Problema do Caixeiro Viajante

Busca exaustiva sequencial e paralela com pthreads

Nomes dos integrantes
Organização de Computadores ▪ UNIVALI

Roteiro e objetivo

Roteiro

1. Problema e modelagem
2. Algoritmo sequencial
3. Paralelização com Pthreads
4. Testes e desempenho
5. Conclusões

- Encontrar a rota de menor custo que visita cada cidade uma vez e retorna à origem.
- Resolver o problema por busca exaustiva *(Brute Force Search)*.
- Implementar uma versão sequencial em C.
- Paralelizar a exploração com threads POSIX (`pthreads`).
- Validar as soluções e comparar o desempenho.

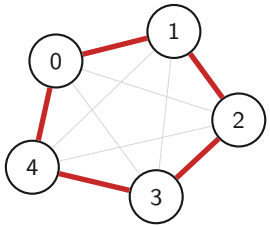
O Problema do Caixeiro Viajante

Dado um conjunto de cidades, representado por um grafo completo ponderado, encontrar o menor ciclo que:

- Visita cada cidade exatamente uma vez;
- Retorna à cidade de origem;
- Minimiza o custo total da rota.

Esclarecendo

As cidades são os vértices e os pesos das arestas representam o custo da viagem.



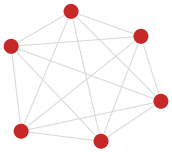
Grafo completo e uma rota possível.

Modelagem Computacional

- N cidades representadas por coordenadas.
- Grafo completo não direcionado e matriz simétrica.
- Distâncias euclidianas pré-calculadas em uma matriz $N \times N$.
- $d(i,j) = d(j,i)$ e $d(i,i) = 0$.

$$d(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

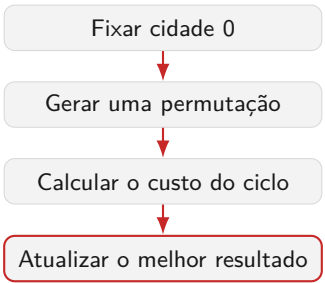
$$C(R) = \sum_{i=1}^{N-1} d(r_i, r_{i+1}) + d(r_N, r_1)$$



Esquema ilustrativo do grafo completo.

Busca Exaustiva

- A cidade 0 é fixada como origem.
- As $N - 1$ cidades restantes são permutadas.
- Cada permutação representa um ciclo candidato.
- O custo é calculado e comparado com o melhor resultado atual.



$P(N) = (N-1)!$ Complexidade temporal: $O((N-1)!) = O(N!)$

Geração das Permutações

```
    if(position == N){
        double actually_cost = dcalculate_cost(distance, route);

        if(actually_cost < *best_cost){
            *best_cost = actually_cost;

            for(unsigned i = 0; i < N; i++){
                best_route[i] = route[i];
            }
        }

        return 1;
    }

    size_t routes_evaluated = 0;
    for(unsigned ci = 1; ci < N; ci++)
    {
        if(!visited[ci]){
            route[position] = ci;
            visited[ci] = true;

            routes_evaluated += dgenerate_permutations(
                distance,
                position + 1,
                route,
                visited,
                best_cost,
                best_route
            );
            visited[ci] = false;
        }

    }

    return routes_evaluated;
}
```

Leitura

- O caso base avalia uma rota completa.
- A cidade escolhida é marcada como visitada.
- A chamada recursiva avança uma posição.
- Ao retornar, a cidade é desmarcada.

Implementação sequencial

- Uma única execução percorre todas as permutações.
- Vetores de rota e de cidades visitadas.
- Recursão/backtracking para construir os caminhos.
- Ao completar uma rota, soma-se também o retorno à origem.
- O menor custo e sua rota são armazenados.

Pseudofluxo

`explorar(rota)`

↓

Escolher próxima cidade

↓

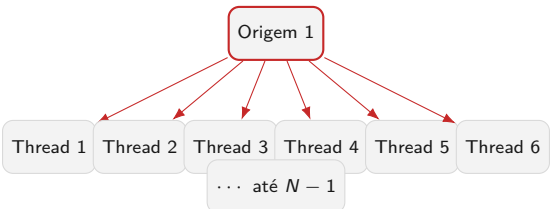
Marcar / recursão

↓

Desmarcar (backtrack)

Divisão da busca entre threads

- A origem permanece fixa na cidade 1 (índice 0).
- São criadas $N - 1$ threads, uma para cada escolha possível da segunda cidade.
- Cada thread explora um subconjunto distinto de permutações.
- Cada worker mantém seu melhor resultado local.



$$(N-1) \times (N-2)! = (N-1)!$$

Criação e junção das threads

```c
void *thread_tsp(void *arg)
{
    ThreadData *data = td_create(arg);
    int route[N];
    bool visited[N] = {false};

    route[0] = 0;
    visited[0] = true;

    route[1] = data->initial_city;
    visited[data->initial_city] = true;

    data->routes_evaluated = dgenerate_permutations(
        data->distance,
        2,
        route,
        visited,
        &data->best_cost,
        data->best_route
    );

    return NULL;
}
```

Worker

Cada worker fixa a origem e sua cidade inicial, explora as permutações restantes e mantém o melhor resultado local.

Criação e junção das threads

```c
size_t run_paralell_tsp(Distance *distance, TSPParallelResult *pr)
{
    for (int i = 0; i < NUM_THREADS; i++) {
        pr->threads_data[i].distance = distance;
    }

    clock_gettime(CLOCK_MONOTONIC, &pr->start);

    int created_threads = 0;
    size_t total_routes = 0;

    for (int i = 0; i < NUM_THREADS; i++) {
        int result = pthread_create(
            &pr->threads[i],
            NULL,
            thread_tsp,
            &pr->threads_data[i]
        );

        if (result != 0) {
            fprintf(stderr,
                    "Erro ao criar thread: %d\n",
                    result);

            for (int j = 0; j < created_threads; j++) {
                pthread_join(pr->threads[j], NULL);
            }

            exit(1);
        }

        created_threads++;
    }
```

Criação

O laço cria $N - 1$ workers. Cada um recebe sua estrutura de dados e inicia a função `thread_tsp`.

Espera e agregação dos resultados

```
    double parallel_best_cost = DBL_MAX;
    int parallel_best_route[N];

    for (int i = 0; i < NUM_THREADS; i++) {
        int result = pthread_join(pr->threads[i], NULL);
        if (result != 0) {
            fprintf(stderr,
                    "Erro ao aguardar thread: %d\n",
                    result);
            exit(1);
        }

        total_routes += pr->threads_data[i].routes_evaluated;

        if (pr->threads_data[i].best_cost < parallel_best_cost) {
            parallel_best_cost = pr->threads_data[i].best_cost;

            for (int j = 0; j < N; j++) {
                parallel_best_route[j] =
                    pr->threads_data[i].best_route[j];
            }
        }
    }

    pr->pbest_cost = parallel_best_cost;
    for (int j = 0; j < N; j++) {
        pr->pbest_route[j] = parallel_best_route[j];
    }

    clock_gettime(CLOCK_MONOTONIC, &pr->end);

    return total_routes;
}
```

Agregação

Cada `pthread_join()` garante que o worker terminou. Depois, a thread principal soma as rotas e seleciona o menor custo global.

Linguagem, bibliotecas e ferramentas

- **Linguagem:** C
- **Paralelismo:** POSIX Threads (`pthread_create`, `pthread_join`)
- **Compilação:** GCC
- **Build:** CMake
- **Análise/visualização:** Python, pandas e Matplotlib
- **Repositório:** `https://github.com/vitorcostadev/tsp-paralell`

Ambiente experimental

Preencher após confirmar a máquina dos testes

Processador / modelo	`[definir]`
Núcleos / threads lógicas	`[definir]`
Memória RAM	`[definir]`
Sistema operacional	`[definir]`
Versão do GCC	`[definir]`
Flags de compilação	`[definir]`

As duas versões devem ser executadas na mesma máquina, com a mesma instância e condições comparáveis. Preencher este slide antes da entrega.

Testes com diferentes valores de N

N	Rotas	Custo ótimo	T_s (s)	T_p (s)	$S = T_s/T_p$	$E = S/(N-1)$
5	24	241.16	0.000003893	0.000295424	0.0132	0.33%
8	5.040	281.28	0.000445391	0.000627304	0.7100	10.14%
10	362.880	300.55	0.041369279	0.023807933	1.7376	19.31%

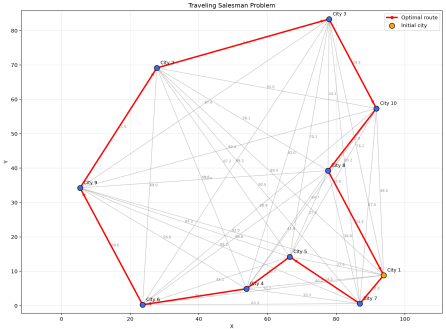
- Cada versão avaliou exatamente $(N-1)!$ rotas.
- Os custos ótimos sequencial e paralelo foram iguais em todos os testes.
- Os testes foram compilados com `-DN=5`, `-DN=8` e `-DN=10`.

Cuidado na interpretação

A igualdade na contagem de rotas evidencia a cobertura; a igualdade do custo e a validação da rota confirmam o resultado.

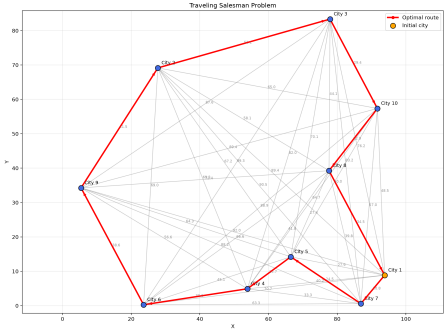
Visualização das soluções

Rota sequencial



$N = 10$

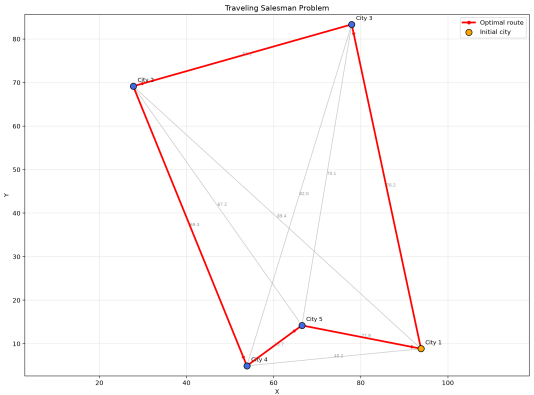
Rota paralela



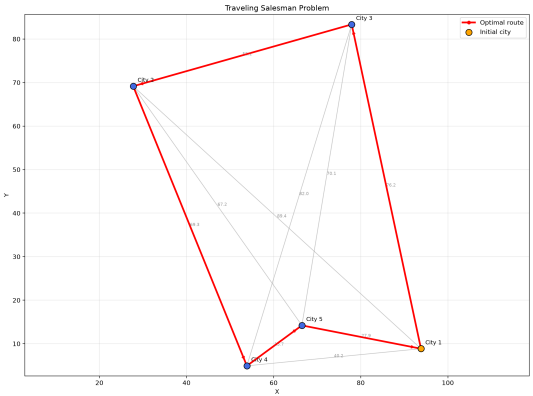
$N = 10$

Rotas encontradas – $N = 5$

Sequencial

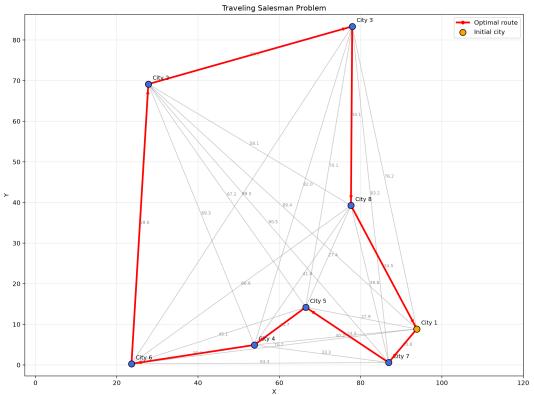


Paralela

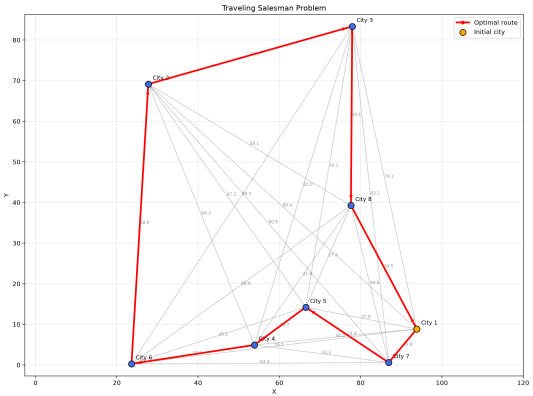


Rotas encontradas – $N = 8$

Sequencial

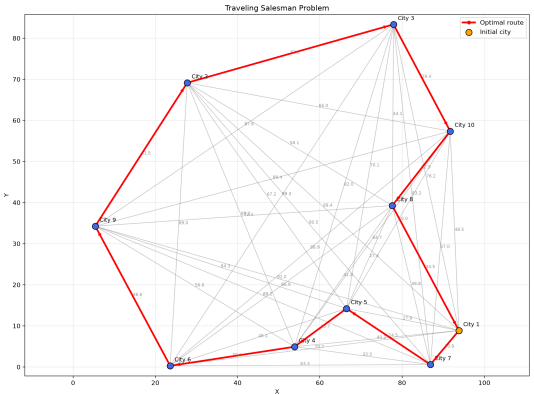


Paralela

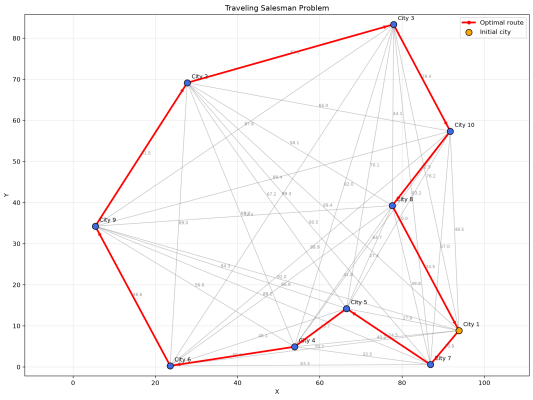


Rotas encontradas – $N = 10$

Sequencial

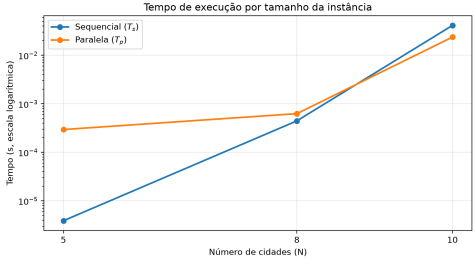


Paralela



Desempenho e speedup

- Para $N = 5$ e $N = 8$, a sobrecarga de criar e sincronizar threads supera o ganho do paralelismo.
- Para $N = 10$, o espaço de busca é maior e foi observado speedup de 1,9915.
- O speedup deve ser calculado por $S = T_s / T_p$, usando tempos medidos.
- Manter a mesma matriz de distâncias ao comparar as duas versões.



Importante

Use somente tempos medidos.

Speedup, eficiência e limitações

$$S(p) = \frac{T_s}{T_p} \qquad E(p) = \frac{S(p)}{p}$$

- Speedup indica quantas vezes a versão paralela foi mais rápida.
- Eficiência relaciona o speedup ao número de workers.
- A criação e a junção das threads também têm custo.
- O código usa $p = N - 1$ workers; isso não significa que existam p núcleos físicos disponíveis.
- O desempenho depende do hardware, do escalonamento e da sobrecarga.

Conclusão

A busca exaustiva permite comparar diretamente a execução sequencial e a divisão do espaço de busca entre threads.

Repositório do projeto

`https://github.com/vitorcostadev/tsp-paralell`

Obrigado!